

Системне Програмування

З використанням мови програмування **Rust**

Основні види програмування

- прикладне
- мобільне (UI)
- фронтенд (UI)
- ігрове
- системне

Деякі класифікації мов програмування

- **Компілятори**

- Assembler
- C, C++, C#, F# ...
- Java, Scala, Haskell
- Rust

- **Інтерпретатори**

- PHP
- Python
- JavaScript
- Ruby

Деякі класифікації мов програмування

- **Статична типізація**

- C, C++, C#, F# ...
- Java, Scala, Haskell
- Rust

- **Динамічна типізація**

- PHP
- Closure
- Python
- JavaScript
- Ruby

Деякі класифікації мов програмування

- **Процедурні**

- Assembly
- Pascal
- C, C++

- **Функціональні**

- Scala
- Haskell
- Closure
- OCaml

- **Об'єктно-орієнтовані**

- PHP
- Java
- Python
- JavaScript
- Ruby

Деякі класифікації мов програмування

- Без збирача сміття

- Assembly
- C, C++
- Rust

- Зі збирачем сміття

- Java
- Haskell
- Rust
- JavaScript
- Python
- Ruby

Деякі класифікації мов програмування

- **Виконуються на віртуальній машині (JVM, CLR,)**

- Java, Scala, Closure
- C#, F#, ...
- Haskell
- JavaScript
- Python
- Ruby

- **Виконуються “на залізі”**

- C, C++
- Pascal
- Rust

Чому з'являються нові мови програмування

- **Змінюються підходи**
 - Процедурні
 - ООП
 - ФП
- **Змінюються потреби**
 - Швидкість розробки
 - Швидкість виконання
 - Коректність
 - Необхідність розширення мови програмування
 - З'являються специфічні задачі
- **З'являються нові targets**
 - mobile
 - browser
 - VM
- ...

Чому потрібно вчити нові мови програмування

- Більшість мов програмування мають унікальні підходи, заради яких вони розроблялися
- Комбінуючи ці підходи можна знайти нові підходи, які можуть зробити розробку більш продуктивно та більш коректно
- Не всі мови програмування є достатньо виразними для вирішення специфічних завдань
- Знаючи декілька підходів можливо набагато швидше та коректніше визначити структуру майбутнього проєкту

Чому Rust?

- Відносно нова мова яка взяла [майже] все найкраще з інших мов
- Гарна екосистема та наявність якісної IDE
- Компілятор (дуууже розумний, з купою підказок стосовно більшості помилок)
- Статична типізація
- Гнучка система типів: підтримка **struct**, **enum**
- Відділення даних (**struct**) від коду (**impl**)
- Відділення декларацій (**trait**) від реалізації (**impl**)
- **No null**
- **No exceptions**
- Extension methods
- Pattern matching / Exhaustiveness checking
- macros / templating
- All targets (native, js, wasm)
- ...

RECAP

1. Базові типи даних

- цілі числа
 - u8 / i8
 - u16 / i16
 - u32 / i32
 - u64 / i64
 - u128 / i128
 - usize / isize
- дрібні
 - f32
 - f64
- bool
- char UTF8
- unit
- never

Назва	Розмір, байтів
i8 / u8	1
i16 / u16	2
i32 / u32	4
i64 / u64	8
i128 / u128	16
isize / usize	2...16
f32	4
f64	8
bool	1
char	1...4
unit	0
never	0

2. Масиви

```
let xs : [i32; 5] = [10, 20, 30, 40, 50];
```

```
let k : i32 = xs[0];  
dbg!(k); // 10
```

```
let k : i32 = xs[5]; // will not compile
```

3. String

```
/// stack  
let data: &str = "initial contents";  
/// move to heap  
let s1: String = data.to_string();  
let s2: String = String::from(data);  
/// initially in heap  
let s3: String = String::from(s: "initial contents");
```


4. Char

```
let c1: char = 'z';  
let c2: char = 'я';  
let c3: char = '你';  
let c4: char = '😊';
```

```
println!("{}", size_of_val(&c1));  
println!("{}", size_of_val(&c2));  
println!("{}", size_of_val(&c3));  
println!("{}", size_of_val(&c4));
```

```
let s5: String = "Hello".to_string();  
for c: u8 in s5.bytes() {  
    print!("{}", c);  
}  
// 72 101 108 108 111
```

```
let hello: &str = "नमस्ते";  
// ['न', 'म', 'स', '्', 'त', 'े']  
// ["न", "म", "स्", "ते"]
```

```
let chars: Chars = hello.chars();  
let bytes: Bytes = hello.bytes();
```

```
dbg!(hello); // "नमस\u{94d}त\u{947}"  
dbg!(&chars); // ['न', 'म', 'स', '\u{94d}', 'त', '\u{947}']  
// ['न', 'म', 'स', '्', 'त', 'े']
```

```
dbg!(&bytes); // [224, 164, 168, 224, 164, 174, 224, 164, 184,  
// -----
```

224, 165, 141,	224, 164, 164,	224, 165, 135]
-----	-----	-----

5. Tuple

```
let x : (i32, f64, String, bool) =  
    (1, 3.14, "Alex".to_string(), true);  
let (a : i32, b : f64, c : String, d : bool) = x  
  
let a : i32 = x.0;  
let b : f64 = x.1;  
let c : String = x.2;  
let d : bool = x.3;
```

6. Named Tuple

```
struct Person(String, i32);  
  
let person: Person = Person("Jim".to_string(), 33);  
  
let Person(name: String, age: i32) = person.clone();  
  
let n: String = person.0;  
let a: i32 = person.1;
```


7. Struct

```
struct Pizza {  
    name: String,  
    size: i32,  
}  
  
let pizza: Pizza = Pizza {  
    name: "Margarita".to_string(),  
    size: 60,  
};  
  
let Pizza { name: String, size: i32 } = pizza  
  
let n: String = pizza.name;  
let s: i32 = pizza.size;
```

8. Enum

$$ax^2 + bx + c = 0$$

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

```
enum Solution {  
    NoRoots,  
    OneRoot(f32),  
    TwoRoots(f32, f32),  
}
```

```
fn solve_quadratic(a: f32, b: f32, c: f32) -> Solution {  
    use Solution::*;  
  
    let d: f32 = b.powi(n: 2) - 4.0 * a * c;  
  
    if d < 0. {  
        NoRoots  
    } else if d == 0. {  
        OneRoot(-b / (2. * a))  
    } else {  
        let dq: f32 = f32::sqrt(d);  
        let a2: f32 = a * 2.;  
        TwoRoots((-b - dq) / a2, (-b + dq) / a2)  
    }  
}
```


9. Pattern matching

```
fn arrays01(xs: &[i32]) {  
    match xs {  
        [] => println!("array is empty"),  
        [x: &i32] => println!("array has one element: {x}"),  
        [x: &i32, y: &i32] => println!("array has two elements: {x}, {y}"),  
        [13, ..] => println!("array starts from number 13: {xs:?}"),  
        [7, 13, ..] => println!("array starts from numbers 7, 13: {xs:?}"),  
        [.., 11, 53] => println!("array ends with numbers 11, 53 {xs:?}"),  
        [.., 53] => println!("array ends with number 53 {xs:?}"),  
        [1, .., 11] => println!("array starts from 1 ends with 11 {xs:?}"),  
        _ => println!("something different {xs:?}"),  
    }  
}
```

10. if ... else

```
if n < 0 {  
    println!("{}", is negative", n);  
} else {  
    println!("{}", is non-negative", n);  
}
```


11. Повторення

```
for n : i32 in 0.. {  
    println!("{}", n);  
}
```

12. Closures (Замикання)

```
fn align_all_v2(data: &[&str], width: u8) -> Vec<String> {  
    let align: fn(&str) -> String =  
        |s: &str|  
            s.to_string() + &* " ".repeat(n: width as usize - s.len());  
  
    data.iter(): impl Iterator<Item=&&str>  
        .map(|x: &&str| align(x)): impl Iterator<Item=String>  
        .collect::<Vec<String>>()  
}
```


13. Блок impl

```
struct Point {  
    x: i32,  
    y: i32,  
}
```

```
impl Point {  
    fn shift(&mut self, dx: i32, dy: i32) {  
        self.x += dx;  
        self.y += dy;  
    }  
  
    fn copy(&self) -> Point {  
        self.clone()  
    }  
}
```

```
impl Point {  
    fn new(x: i32, y: i32) -> Self {  
        Point { x, y }  
    }  
  
    fn random() -> Self {  
        let mut rt: ThreadRng = rand::thread_rng();  
        let x: i32 = rt.gen_range(range: 5..50);  
        let y: i32 = rt.gen_range(range: 3..100);  
        Point::new(x, y)  
    }  
}
```

```
impl Point {  
    const CENTER: Point = Point { x: 0, y: 0 };  
}
```

14. Traits

```
trait Animal {  
    fn sound(&self) -> String;  
}
```

```
struct Sheep(String);  
struct Cow;  
  
impl Animal for Sheep {  
    fn sound(&self) -> String { format!("I'm {:?}, maa-maa", self) }  
}  
  
impl Animal for Cow {  
    fn sound(&self) -> String { String::from(s: "moo-moo") }  
}  
  
impl Animal for i32 {  
    fn sound(&self) -> String { format!("I'm i32 and hold value: {self}") }  
}
```


15. Колекції

- Vec
- VecDeque
- LinkedVist
- HashSet
- HashMap
- BTreeSet
- BTreeMap

16. Ітератори

```
let mm: Months = Months::winter();  
  
for m: String in mm {  
    println!("{}", m);  
}
```

```
December  
January  
February
```


17. Значення та посилання

Rust — це нізькорівнева мова програмування (як і мова C), тому операціям з пам'яттю приділяється багато нюансів.

Rust розділяє значення та посилання, та має відповідний синтаксис для роботи з ними (синтаксис такий самий як у мові C)

```
let x : i32 = 5;  
  
let px : &i32 = &x;  
  
let x2 : i32 = *px;
```

18. Input/Output

```
#[test]
fn test2_write_bin_file() -> std::io::Result<()> {
    let file : File = File::create(path: "output.bin"?);
    let mut buf : BufWriter<File> = BufWriter::new(file);
    buf.write_all(buf: &[0x42, 0x55, 0x53, 0x59])?;
    Ok(())
}
```


New Topics

- Standard enums with examples
- Standard traits with examples
- Error Handling
- Memory. Ownership, Box, Rc, Arc
- Recursion
- Generics
- Functional Programming
- Serialization, Json
- WebServer
- Database, SQL
- Concurrency

**Код з лекцій,
презентації Keypnote,
PDF-файли
знаходяться на GitHub:**

<https://github.com/djnzx/rust-course>